

Assignment 1: Step Counting

Overview

You will improve a step-counting algorithm on the class dataset you helped build. This assignment is done in **groups of three** for the analysis work; data collection happens via the semester-long SensorLogger Studies you joined in A0.

Use of AI coding assistants (Claude Code, Cursor, Copilot, ChatGPT) is **permitted** on the code, exploration report, and writeup. The individual assignment quiz is closed-resource and graded on whether you understood what you were doing.

What's in this repo

```
├─ RUBRIC.md                # assignment instructions and grading
├─ requirements.txt          # pip install -r requirements.txt
├─ solution.py              # baseline counter (you improve this) -> exposes count_steps(csv_file)
├─ step_counting_and_analysis.ipynb  # playground for plots, failure analysis
├─ data/
│   ├── sample_walk_100steps.csv      # sanity-check sample (100 known steps)
│   └─ Class_Data/
│       ├── manifest.csv              # file, true_steps, placement (230 recordings)
│       └─ sensor_data/
│           ├── accelerometer CSVs
│           └─ accelormeter_<zipfile_name>.csv # original ZIP name without .zip
├─ utils/
│   ├── visualization.py             # plot_with_peaks, zoom_plot, etc.
│   └─ eval.py                      # local evaluator (per-file results and overall summaries)
```

Data collection

Data collection happens via the **CS328 Steps Study** you joined in A0. Every recording needs an exact step count and the phone placement in the questionnaire.

You're encouraged to identify and contribute additional **edge cases** beyond your assigned scenarios — anything you think might be a real world case where steps would be difficult to detect.

Analysis

The class corpus as of **Sep 20, 2026** is in data/Class_Data/. Your job is to **improve solution.py** so it works better on the corpus, and to **document your exploration** systematically in the report.

Use `data/Class_Data/manifest.csv` to select recordings. It lists **230 accelerometer CSVs** in `data/Class_Data/sensor_data/`, with one row per file:

Column	Meaning
file	Path to the sensor CSV, relative to the repository root
true_steps	Ground-truth step count from the recording questionnaire (NumSteps)
placement	Phone location from the recording questionnaire

Each sensor CSV contains `time`, `seconds_elapsed`, `z`, `y`, and `x`. `time` is in nanoseconds, `seconds_elapsed` is in seconds, and `x`, `y`, and `z` are the accelerometer axes.

What `solution.py` must do

- Export `count_steps(csv_file: str) -> int` — given a path to a recording CSV, return the integer step count.
- Optionally support `count_steps(csv_file, return_debug=True) -> (int, debug_dict)` for plotting in the notebook. Debug dict can include `peaks`, `signal`, `df`, anything else you want to visualize.

Baseline The provided `solution.py` combines the three axes into an acceleration magnitude, applies a centered five-sample moving average, and calls `scipy.signal.find_peaks` with fixed `height=10` and `distance=50`. These are starting parameters, not validated settings for every recording. Measure where the baseline succeeds or fails, then improve it. Peak distance and moving-average window sizes are in samples; use the timestamps to relate them to seconds.

Two design axes A1 has **two independent design choices**.

Axis	What you choose	Example parameters
Noise removal / filtering	Filter family + cutoff/window	low-pass cutoff Hz, MA window, median vs mean, FFT cutoff, Butterworth order
Step detection / peak finding	Algorithm + parameters	<code>find_peaks(height, prominence, distance, width)</code> , or autocorrelation, zero-crossings

A different filter changes what peak parameters work; pick a filter and detection method that work together.

You have full freedom on approach (time domain, frequency domain, adaptive thresholds, etc.) — just be clear in the report about what you chose.

Run the evaluator locally Run commands from the repository root:

```
pip install -r requirements.txt
python -m utils.eval --manifest data/Class_Data/manifest.csv --report-dir reports/
```

The evaluator reads the three-column manifest and writes `reports/results.csv` and `reports/results.md`. These contain per-file results (detected vs. true steps, absolute and relative error, and whether the prediction is within $\pm 20\%$) and overall summaries. Failed runs count as failures in pass rates and are excluded from mean and median error. Use the placement column or your documented recording categories for category comparisons. A nonzero exit status can also mean that some recordings exceeded the $\pm 20\%$ threshold; consult the report to distinguish prediction failures from a code error.

How we evaluate your code Your final `solution.py` is run on the **full class corpus** at submission time. Code grading uses **three performance tiers**, described under Assessment.

Deliverables

Submit on Canvas by Oct 4. One submission per group:

1. `solution.py` — your improved counter Same interface as the baseline. Documented inline where the choices matter.

2. `exploration_report.pdf` — your systematic exploration (the depth artifact) Use the templates below.

§1: Filter design space exploration

For each scenario category your group worked on (~6 rows):

Scenario	Filter family	Key parameter	Why this filter helps this scenario
(e.g., Trouser pocket)	Butterworth low-pass	cutoff = 3 Hz	Pocket bounce is high-frequency relative to gait
...	...	...	...

§2: Peak detection design space exploration

For the same scenarios:

Scenario	Algorithm	height	prominence	distance	width	Rationale
(e.g., Trouser pocket)	<code>find_peaks</code>	—	0.4	0.4 s	5	Lower prominence catches the smaller peaks; wider min-distance suppresses pocket-bounce double-counts
...	...	...	...	...	...	...

State the units for every parameter. For `find_peaks`, distance and width are measured in samples; convert time-based choices using each recording's sampling rate.

§3: Your universal solution

- The filter configuration and step-detection parameters or adaptive rules in your final `solution.py` (one algorithm that runs across all scenarios/placements without manual per-file tuning).
- One paragraph: how you decided what to fix universally vs. what to sacrifice.

§4: Generalization gap

Scenario	Per-scenario best accuracy	Universal accuracy	Gap (percentage points)	Where the gap comes from (filter / peak detection / both)
(e.g., Trouser pocket)	78%	64%	-14	Filter — universal lowpass at 5 Hz passes the pocket-bounce harmonics that our per-scenario 3 Hz lowpass cut
(e.g., Stair descent)	71%	58%	-13	Peak detection — universal prominence (0.6) misses the smaller heel-strike peaks; per-scenario used 0.4
...	...	...	...	...

Define the accuracy metric used in your table, and compare the per-scenario and universal configurations on the same recordings. Compute the gap as universal accuracy minus per-scenario best accuracy. Attribute each gap to filtering, step detection, or both, and support the explanation with your observations.

§5: Top failure modes + edge cases

For each of your **top 2 failure modes**:

- The scenario or recording (cite a specific filename or trace from the class corpus).
- A figure: a plot of the trace, with the failure annotated (false positives, missed steps, etc.). Embed the figure or link to the notebook cell that generates it.
- 1-2 sentences: the physical mechanism causing the failure.
- 1 sentence: is this primarily a **filter** problem or a **peak detection** problem?
- 1 sentence: one specific algorithmic idea that would help (whether or not you implemented it).

Plus: identify at least **one unusual edge case** from the class corpus that surprised you (a recording where the failure mechanism wasn't obvious from the scenario label). Show the trace, describe what went wrong, and explain why the label alone didn't predict it.

Example of a §5 entry (illustrative — the outcomes and physical explanations below are hypothetical, not verified annotations for these files; fill in with your own findings):

Failure Mode 1: Hand-held

- Recording: `accelormeter_2026-09-11_01-01-04.csv`
- Figure: see notebook cell 23 — annotated trace showing extra peaks around 1 Hz overlaid on the gait signal.
- Physical mechanism: arm gestures during talking add a ~1 Hz periodic motion on top of the ~2 Hz gait. After our universal lowpass at 5 Hz, both pass through and combine into mixed peaks that our detector counts as steps.
- Filter or peak detection? Primarily a **filter** problem — both frequencies pass our cutoff.

- Algorithmic idea: detect dominant periodicity per window and apply an adaptive bandpass around the detected gait frequency.

Edge case (unexpected):

- Recording: `accelormeter_2026-09-10_01-27-25.csv` — labeled “trouser pocket,” but our counter only got 15 of 30 true steps.
- Possible explanation: a heavy backpack may have damped vertical motion, reducing peak amplitude below our prominence threshold.
- Why the label alone was insufficient: the questionnaire captured placement (trouser pocket), but not load (such as a heavy backpack).

3. writeup.pdf — half-page reflection Cover, briefly:

- What was the single most important change you made vs the baseline? (~2 sentences)
- What surprised you most when you ran your code on classmates’ data? (~2 sentences)
- What would you try next, with 4 more hours? (~1 sentence)

4. Contribution statement One line: who did what.

Assessment

48 points. Mix of group-level (artifact) and individual (assignment quiz). The general concept quiz on signal processing fundamentals is graded separately as part of the class quiz series, not under A1.

Group-level (36 pts)

- **Data collection completeness (12 pts)** — All assigned placements present in the class corpus by deadline, with questionnaire fully filled. Each group member contributed at least 10 recordings.
- **Code grading (12 pts)** — broad-bucket on the full class corpus:

Tier	Criterion	Points
Failed	Code crashes, or aggregate metric is worse than the baseline	0
Working	Code runs cleanly, aggregate metric $\geq$ baseline	9
Improved	Working AND $\geq 10\%$ relative improvement over baseline on at least one major scenario category	12

- **Exploration report (10 pts)** — pass/fail on the 5 required sections being present and substantive:
 - §1 filter table populated (2 pts)
 - §2 peak detection table populated (2 pts)
 - §3 universal solution + paragraph (1 pt)

- §4 generalization gap table with attributions (2 pts)
- §5 top failure modes with figures + edge case (3 pts)
- **Writeup + contribution statement (2 pts)** — present, half-page or less, all three reflection bullets addressed.

We deliberately use broad buckets, not percentile ranking. Mobile health metrics are noisy; rewarding the difference between 89.2% and 87.6% is statistical theater. Depth of understanding is graded at the assignment quiz, not here.

Individual (12 pts)

- **Assignment quiz (12 pts)**
-

On using AI

You may use AI coding assistants on `solution.py`, the exploration report, and the writeup. However, note that **the assignment quiz tests whether you, specifically, engaged with the work**. Questions on the quiz directly reference specific sections of your group's submitted exploration report — vague answers that don't match the report will be flagged.

Late Policy

- Each group has **5 late days**, usable across all assignments for final code submissions.
 - Late submission penalty: –10% per day after the 5 free late days.
 - **Assignment quiz is individual; no late days apply** except for documented emergencies.
 - Significant contribution imbalance (as reported) → individual grading may be triggered.
-